

Module 5: Internet of Things

IoT Physical Devices

What is an IoT device, Exemplary device: Raspberry Pi, Linux on Raspberry Pi -interfaces - Programming Raspberry Pi with Python-Other IoT Devices.

What is an IoT Device?

The “Thing” in Internet of Things (IoT) is any object that has a unique identifier and which can send/ receive data (including user data) over a network. IoT devices are connected to the internet and send information about themselves or about their surroundings over a network or allow actuation upon the physical entities/ environment around them remotely. Some examples of IoT devices are:

1. A home automation device that allows remotely monitoring the status of appliances and controlling the appliances.
2. An industrial machine which sends information about its operation and health monitoring data to a server.
3. A car which sends information about its location to a cloud-based service.
4. A wireless-enabled wearable device that measures data about a person such as the number of steps walked and sends the data to a cloud-based service.

Basic building blocks of an IoT Device

A IoT device can consist of a number of modules based on functional attributes, such as:

1. **Sensing:** Sensors can be either on-board the IoT device or attached to the device. IoT device can collect various types of information from the on-board or attached sensors such as temperature, humidity, light intensity, etc. The sensed information can be communicated either to other devices or cloud-based servers/ storage.
2. **Actuation:** IoT devices can have various types of actuators attached that allow taking actions upon the physical entities in the vicinity of the device. For example, a relay switch connected to an IoT device can turn an appliance on/off based on the commands sent to the device.
3. **Communication:** Communication modules are responsible for sending collected data to other devices or cloud-based servers/storage and receiving data from other devices and

commands from remote applications.

4. **Analysis & Processing:** They are responsible for making sense of the collected data.

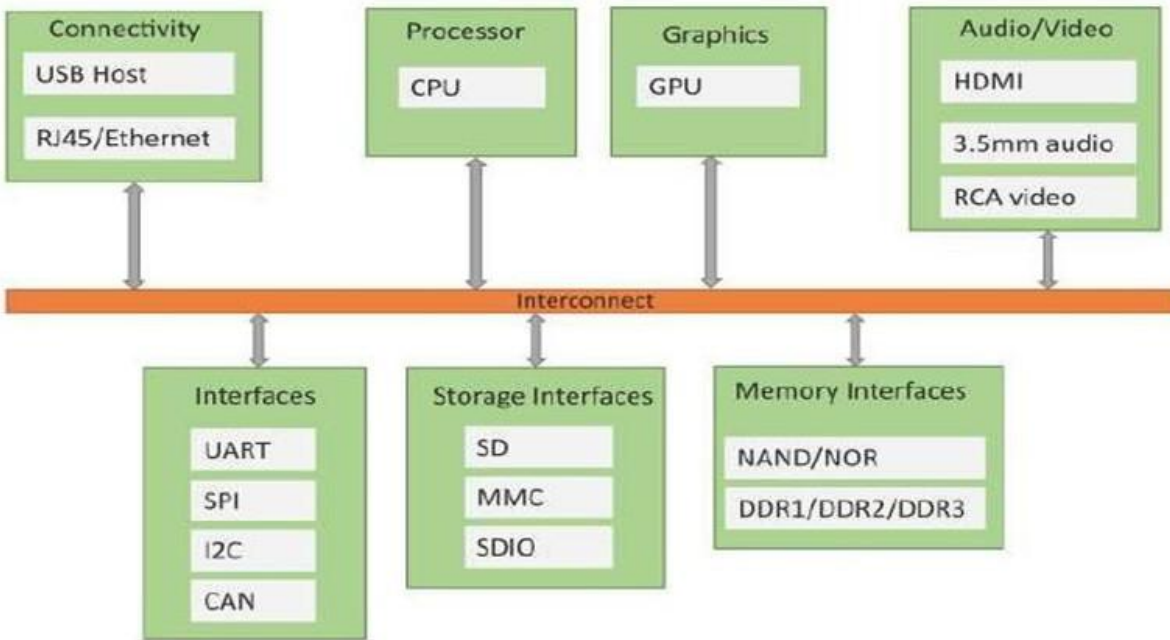


Figure 1: Block Diagram of an IoT Device

The figure represents the **generic building blocks of a Single Board Computer (SBC)-based IoT device**. This is similar to how a Raspberry Pi or other SBCs are structured. It shows how different subsystems (processor, graphics, memory, storage, connectivity, and interfaces) are connected via a common **interconnect bus**.

1. Connectivity

- **USB Host:** Allows connection of external USB devices such as keyboards, storage devices, and cameras.
- **RJ45/Ethernet:** Provides wired internet or LAN connectivity for high-speed communication.

2. Processor

- **CPU (Central Processing Unit):**
 - Executes instructions and performs the main computational tasks.

- Handles system logic, scheduling, and control operations.

3. Graphics

- **GPU (Graphics Processing Unit):**
 - Handles rendering of images, video, and graphical data.
 - Enhances performance for multimedia and visualization tasks.

4. Audio/Video Interfaces

- **HDMI:** High-definition video and audio output for displays.
- **3.5mm Audio:** Standard jack for headphones/speakers.
- **RCA Video:** Analog video output for older displays.

5. Interfaces

- **UART (Universal Asynchronous Receiver/Transmitter):** Serial communication with sensors, microcontrollers.
- **SPI (Serial Peripheral Interface):** High-speed communication with external peripherals like ADC/DAC, sensors.
- **I2C (Inter-Integrated Circuit):** Multi-device communication on a two-wire bus.
- **CAN (Controller Area Network):** Used in automotive and industrial applications for robust communication.

6. Storage Interfaces

- **SD (Secure Digital card):** For expandable storage.
- **MMC (MultiMedia Card):** Storage medium for embedded devices.
- **SDIO (Secure Digital Input Output):** Allows interfacing of Wi-Fi/Bluetooth modules via SD card slot.

7. Memory Interfaces

- **NAND/NOR Flash:** Non-volatile memory for storing firmware, OS, and data.
- **DDR1/DDR2/DDR3:** Dynamic RAM for fast, temporary data access and execution.

8. Interconnect (Bus)

- The backbone that connects CPU, GPU, memory, storage, and interfaces.
- Ensures communication and data transfer between components.

Exemplary Device: Raspberry Pi

Raspberry Pi is a low-cost mini-computer with the physical size of a credit card. It runs various flavors of Linux and can perform almost all tasks that a normal desktop computer can do. In addition to this, Raspberry Pi also allows interfacing sensors and actuators through the general purpose I/O pins. Since Raspberry Pi runs Linux operating system, it supports Python “out of the box”.

About the Board

The Raspberry Pi board with the various components/peripherals is labeled as follows:

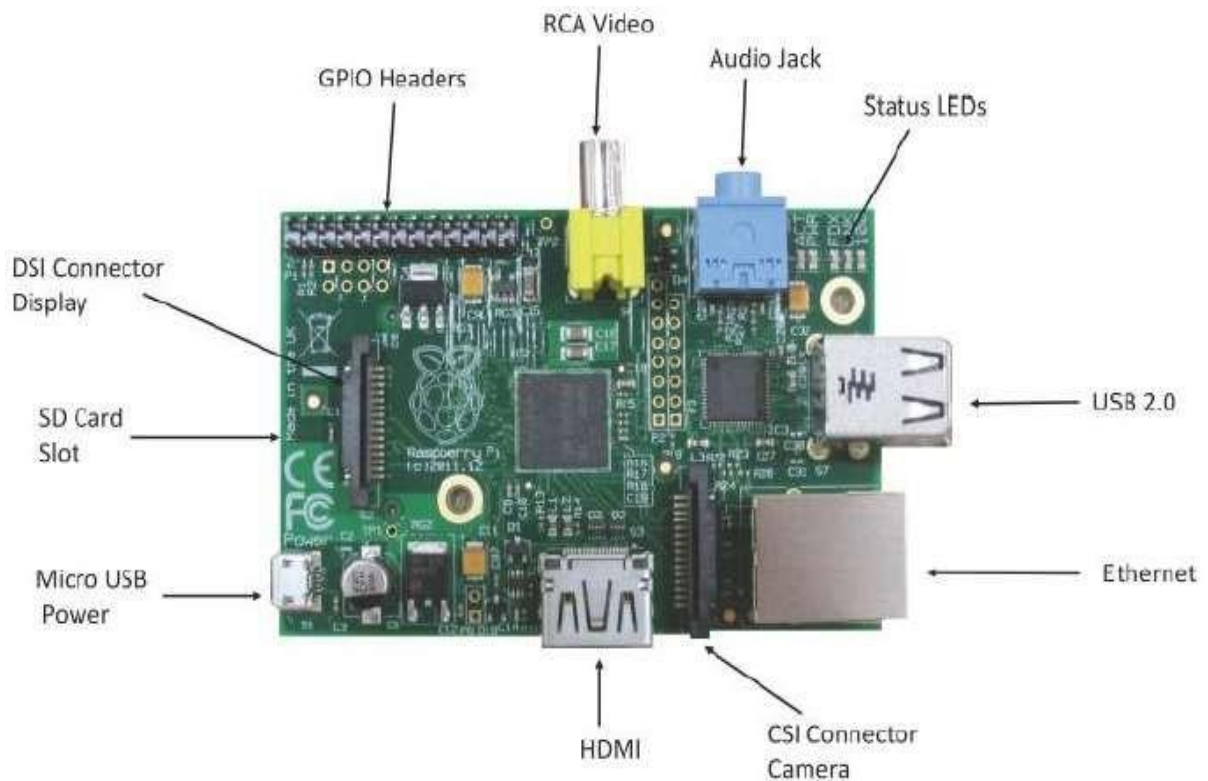


Figure 2: Raspberry Pi Board

- **Processor & RAM:** Raspberry Pi is based on an ARM processor. The latest version of Raspberry Pi (Model B, Revision 2) comes with 700 MHz Low Power ARM1176JZ-F processor and 512 MB SDRAM.
- **USB Ports:** Raspberry Pi comes with two USB 2.0 ports. The USB ports on Raspberry Pi can provide a current upto 100mA. For connecting devices that draw current more than 100mA, an external USB powered hub is required.
- **Ethernet Ports:** Raspberry Pi comes with a standard RJ45 Ethernet port. You can connect an Ethernet cable or a USB Wifi adapter to provide Internet connectivity.
- **HDMI Output:** The HDMI port on Raspberry Pi provides both video and audio output. You can connect the Raspberry Pi to a monitor using an HDMI cable. For monitors that have a DVI port but no HDMI port, you can use an HDMI to DVI adapter/cable.
- **Composite Video Output:** Raspberry Pi comes with a composite video output with an RCA jack that supports both PAL and NTSC video output. The RCA jack can be used to connect old televisions that have an RCA input only.
- **Audio Output:** Raspberry Pi has a 3.5mm audio output jack. This audio jack is used for providing audio output to old televisions along with the RCA jack for video. The audio quality from this jack is inferior to the HDMI output.
- **GPIO Pins:** Raspberry Pi comes with a number of general-purpose input/output pins.

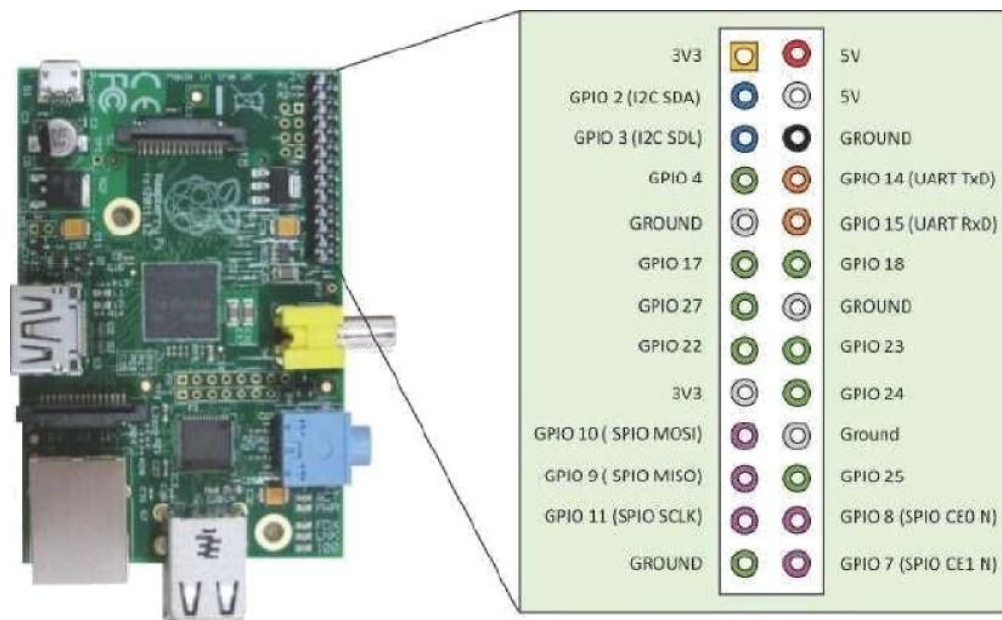


Figure 3: shows the Raspberry Pi GPIO headers

There are four types of pins on Raspberry Pi - true GPIO pins, 12C interface pins, SPI interface pins and serial Rx and Tx pins.

- **Display Serial Interface (DSI):** The DSI interface can be used to connect an LCD panel to Raspberry Pi.
- **Camera Serial Interface (CSI):** The CSI interface can be used to connect a camera module to Raspberry Pi.
- **Status LEDs:** Raspberry Pi has five status LEDs. Table 7.1 lists Raspberry Pi status LEDs and their functions.

Table 1: Raspberry Pi Status LEDs

Status LED	Function
ACT	SD Card Access
PWR	3.3V Power is present
FDX	Full Duplex LAN connected
LNK	Linux Network Activity
100	100Mbit LAN Connected

- **SD Card Slot:** Raspberry Pi does not have a built-in operating system and storage. You can plug-in an SD card loaded with a Linux image to the SD card slot. Appendix-A provides instructions on setting up New Out-of-the-Box Software (NOOBS) on Raspberry Pi. You will require atleast an 8GB SD card for setting up NOOBS.
- **Power Input:** Raspberry Pi has a micro-USB connector for power input.

Linux on Raspberry Pi

Raspberry Pi supports various flavors of Linux including:

1. **Raspbian:** Raspbian Linux is a Debian Wheezy port optimized for Raspberry Pi. This is the recommended Linux for Raspberry Pi.
2. **Arch:** It is an Arch Linux port for ARM devices.
3. **Pidora:** It is a Fedora Linux optimized for Raspberry Pi.
4. **RaspBMC:** It is an XBMC media-center distribution for Raspberry Pi.
5. **OpenELEC:** It is a fast and user-friendly XBMC media-center distribution.
6. **RISC OS:** It is a very fast and compact operating system.

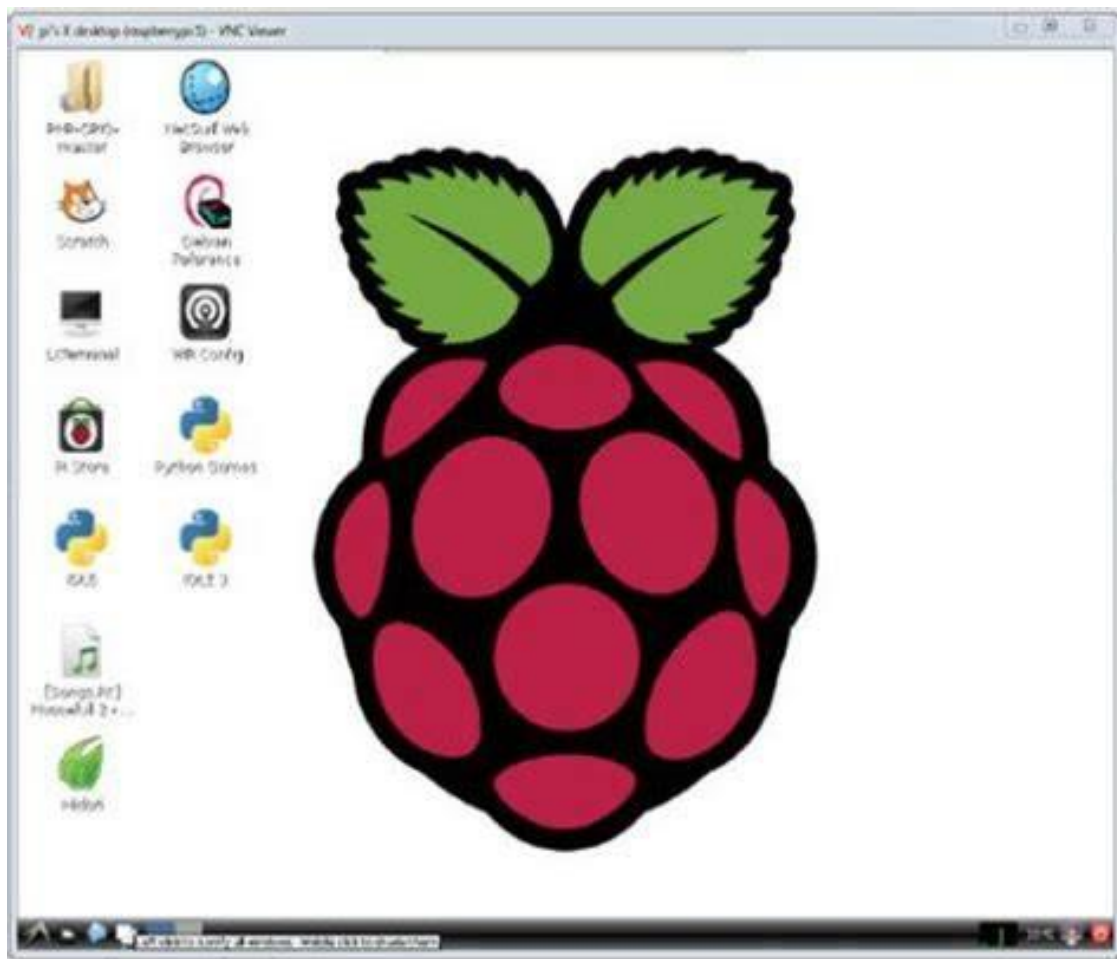
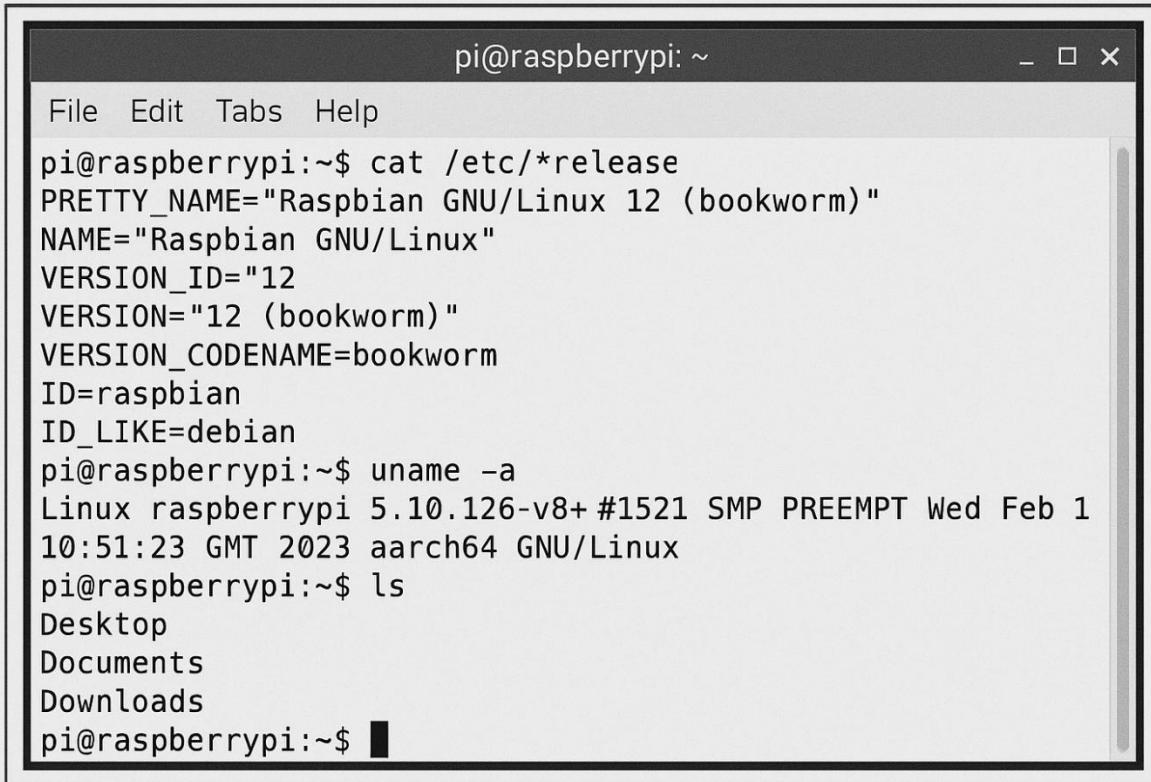


Fig 4: Raspbian Linux desktop

This figure shows the Raspbian (Raspberry Pi OS) desktop environment, which is the default graphical interface used on Raspberry Pi devices. The desktop features a simple and lightweight layout, with application shortcuts placed on the left side for quick access to tools such as the web browser, programming environments like Python and Scratch, and system configuration utilities. The center of the screen prominently displays the Raspberry Pi logo, representing the operating system and hardware platform. At the bottom, a taskbar provides essential functions including the main menu, file manager, terminal, system notifications, network status, audio control, and clock. Designed for beginners as well as developers, this desktop environment offers an easy-to-use interface for coding, internet browsing, and managing Raspberry Pi hardware and software settings.



```
pi@raspberrypi: ~
File Edit Tabs Help
pi@raspberrypi:~$ cat /etc/*release
PRETTY_NAME="Raspbian GNU/Linux 12 (bookworm)"
NAME="Raspbian GNU/Linux"
VERSION_ID="12"
VERSION="12 (bookworm)"
VERSION_CODENAME=bookworm
ID=raspbian
ID_LIKE=debian
pi@raspberrypi:~$ uname -a
Linux raspberrypi 5.10.126-v8+ #1521 SMP PREEMPT Wed Feb 1
10:51:23 GMT 2023 aarch64 GNU/Linux
pi@raspberrypi:~$ ls
Desktop
Documents
Downloads
pi@raspberrypi:~$
```

Fig 5: Console on Raspberry Pi

The figure shows a Linux console running on a Raspberry Pi, which provides a command-line interface for interacting directly with the Raspberry Pi OS. The prompt displayed as `pi@raspberrypi: ~$` confirms that the system is using the default user account *pi* on a Debian-based Linux distribution. Through this console, users can execute essential commands such as viewing system information, managing files, installing software, and configuring hardware interfaces. Commands like `cat /etc/*release` and `uname -a` display details about the operating system version, Linux kernel, and architecture, all of which indicate that the device is running Raspbian/Raspberry Pi OS, a lightweight Linux distribution optimized for ARM hardware.

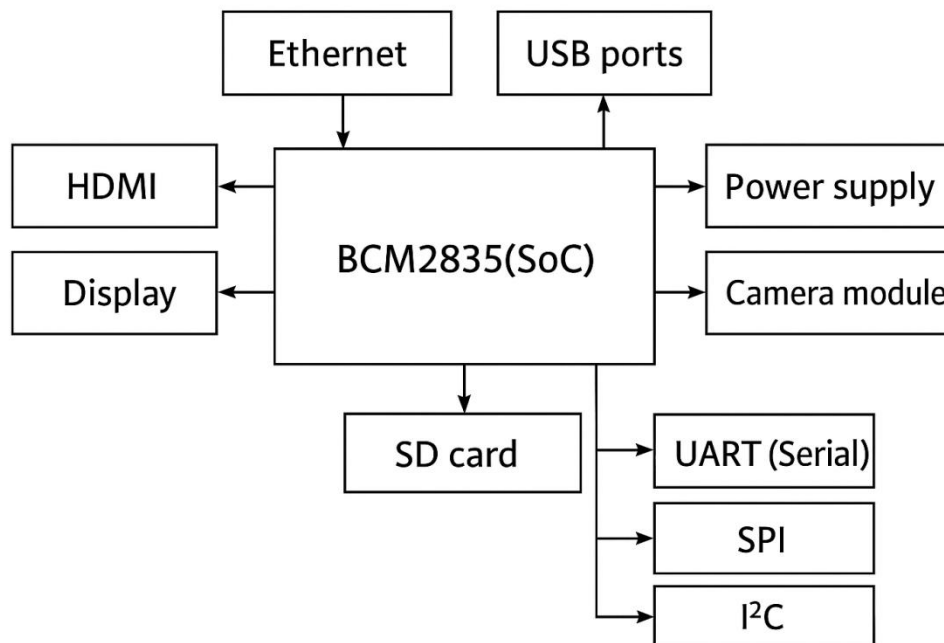
Using the console is one of the most powerful ways to control a Raspberry Pi because it offers full access to the underlying Linux system without needing a graphical interface. Developers and students often rely on this terminal environment to compile programs, modify configuration files, monitor system performance, and automate tasks using shell scripts. The simplicity and efficiency of the Linux command line make it ideal for embedded systems like the Raspberry Pi, where direct hardware access and precise control are important for projects, learning, and experimentation.

The below table represents the frequently used commands on Raspberry Pi

Command	Function	Example
cd	Change directory	cd/home/pi
cat	Show file contents	Cat file.txt
ls	List files and folders	ls/home/pi
locate	Search for a file	Locate file.txt
lsusb	List USB devices	lsusb
pwd	Print name of present working	pwd
mkdir	Make directory	mkdir/home/pi/new
mv	Move (rename) file	Mv sourceFile.txt, destinationFile.txt
rm	Remove file	rm file.txt
reboot	Reboot device	sudo reboot
shutdown	Shutdown device	sudo shutdown _h now
grep	Print line matching a pattern	grep -r “pi” /home/
df	Report file system disk space usage	df -Th
ifconfig	Configure a network interface	ifconfig
netstat	Print network connections, routing tables, interface statistics	netstat -lntp
tar	Extract/create archive	Tar -XZf foo.tar.gz
wget	Non-interactive network downloader	Wget http://example.com/file.tar.gz

Table 2: Raspberry Pi frequently used commands

Raspberry Pi -interfaces



Raspberry Pi Interface Block Diagram

Raspberry Pi has serial, SPI and I²C interfaces for data transfer along with several other communication and peripheral interfaces.

1. Serial:

- The Raspberry Pi includes a hardware UART (Universal Asynchronous Receiver–Transmitter) on its GPIO header.
- It uses only **two data pins**: **TxD (Transmit)** sends data from the Pi to a peripheral, and **RxD (Receive)** receives data from the peripheral into the Pi.
- Communication is **asynchronous**, meaning there is no separate clock line; both ends must be configured for the same baud rate (e.g., 9600 bps, 115200 bps).
- The link is **full-duplex**, so data can be sent and received simultaneously on separate lines.
- Because it is simple and requires minimal wiring, UART is widely used to:
 - Connect microcontrollers such as Arduino to Raspberry Pi.
 - Communicate with serial sensors and GPS modules.
 - Provide a serial console for debugging or headless access.

2. SPI (Serial Peripheral Interface):

- Serial Peripheral Interface (SPI) is a synchronous serial data protocol used for communicating with one or more peripheral devices.
- In an SPI connection, there is one master device and one or more peripheral devices. There are live pins on Raspberry Pi for SPI interface:
 - a. MISO (Master in Slave Out): Master line for sending data to the peripherals.
 - b. MOSI (Master Out Slave In): Slave line for sending data to the master.
 - c. SCK (Serial Clock): Clock generated by master to synchronize data transmission
 - d. CEO (Chip Enable 0): To enable or disable devices.
 - e. CEO (Chip Enable 1): To enable or disable devices.
- SPI supports full-duplex transfer (simultaneous send/receive) and much higher speeds than UART, which makes it well-suited to:
 - Driving graphical or TFT/OLED displays.
 - Communicating with high-speed sensors, ADC/DAC converters, or flash memory chips.
 - Situations where low latency and high throughput are needed.

3. I2C (Inter-Integrated Circuit):

- I²C is a **two-wire synchronous bus** designed for short-distance communication between a master and multiple slaves using only two lines regardless of the number of devices.
- The Raspberry Pi provides:
 - **SDA (Serial Data)**: a bidirectional line that carries addresses and data between the master and slaves.
 - **SCL (Serial Clock)**: a clock line generated by the master to synchronise all data transfers.
- Every device on the I²C bus has a **unique address**, allowing the master to select one device at a time while all others ignore the message.
- Because only two wires are needed, wiring is very simple and it is ideal for connecting multiple low- or moderate-speed peripherals.

- The Raspberry Pi includes built-in pull-up resistors on SDA and SCL, which are necessary for the bus to function.
- Common uses include:
 - Reading data from temperature, pressure, humidity, or light sensors.
 - Controlling real-time clock (RTC) modules.
 - Expanding GPIO pins with I/O expander chips.
 - Driving small character or graphic displays.

4. Ethernet

- Provides **wired LAN connectivity** using an RJ-45 port.
- Offers **stable, high-speed, and low-latency** network communication.
- Not affected by Wi-Fi interference, making it reliable for continuous data transfer.
- Commonly used for **IoT gateways, servers, NAS systems, and real-time applications**.
- Supports both **DHCP and static IP addressing**.

5. USB Ports

- Standard **USB 2.0/3.0 ports** depending on the Raspberry Pi model.
- Used to connect **keyboard, mouse, storage devices, Wi-Fi adapters**, etc.
- Supports **plug-and-play peripherals** for easy expansion.
- Can power small USB devices like sensors or microcontrollers.
- Makes the Pi function like a **full computer system** with peripheral support.

6. HDMI

- Main interface for **digital video and audio output**.
- Connects the Pi to **monitors, TVs, and projectors**.
- Supports **Full HD** and higher resolutions (model dependent).
- Essential for GUI-based applications like **Raspbian Desktop**, media centers, and dashboards.
- Enables visualization of **camera feeds, UI, and real-time data**.

7. Display (DSI)

- Stands for **Display Serial Interface (DSI)**.
- Designed for connecting **official Raspberry Pi touchscreens and LCD displays**.
- Provides a **high-speed direct display connection**, separate from HDMI.

- Suitable for **embedded systems, kiosks, and portable devices** needing a compact screen.
- Allows touch functionality with minimal wiring.

8. Camera (CSI)

- Stands for **Camera Serial Interface (CSI)**.
- Interface for connecting **Raspberry Pi Camera Module**.
- Supports **high-bandwidth** image and video capture with low latency.
- Used for **computer vision, robotics, surveillance, and AI projects**.
- More efficient than USB cameras due to direct sensor-to-processor communication.

9. SD Card

- Acts as the **main storage device** of the Raspberry Pi.
- Contains the **bootloader, operating system, applications, and user data**.
- Allows easy OS swapping—insert another SD card to change environments.
- Stores logs, sensor data, project files, and databases for IoT applications.
- Essential for **booting and running the Raspberry Pi**.

10. GPIO Ports

- Stands for **General Purpose Input/Output**.
- Pins can be configured as **input** (read sensor data) or **output** (control LEDs, motors).
- Provides hardware access to **UART, SPI, and I²C protocols**.
- Enables the Pi to interface with **sensors, actuators, relays, and other electronics**.
- Core component for IoT, automation, robotics, and embedded system projects.

11. Power Supply

- Raspberry Pi uses **5V DC** power via **micro-USB or USB-C** (model-dependent).
- Powers the SoC, ports, and all connected peripherals.
- Stable power is crucial to avoid **reboots, SD card corruption, or performance drops**.
- Typically requires **2A–3A** depending on the workload.
- Includes voltage regulation and protection circuitry for safe operation.

Programming Raspberry Pi with Python

Raspberry Pi runs Linux and supports Python out of the box. Therefore, any python program can be run the same way as they are run on a normal computer. However, it is the general purpose input/output capability provided by the GPIO pins on Raspberry Pi that makes it a useful device for Internet of Things. The user can interface a wide variety of sensor and actuators with Raspberry Pi using the GPIO pins and the SPI, I2C and serial interfaces. Input from the sensors connected to Raspberry Pi can be processed and various actions can be taken, for instance, sending data to a server, sending an email, triggering a relay switch.

Controlling LED with Raspberry Pi

The basic example of controlling an LED from Raspberry Pi is shown in the figure below:

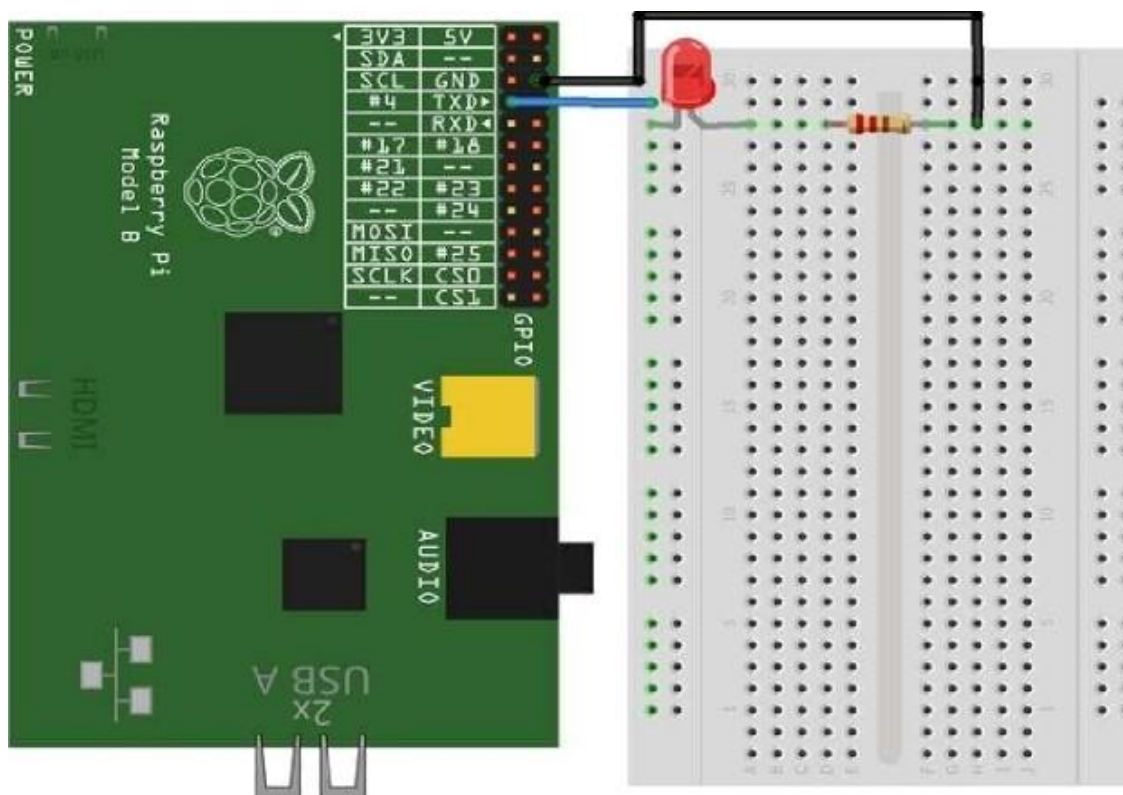


Fig 4: Controlling LED with Raspberry PI

The code to turn the LED on/off from command line is presented as follows:

Switching LED on/off from Raspberry Pi console

```
$echo 18 > /sys/class/gpio/export
```

```
$cd /sys/class/gpio/gpio18
```

```
$set pin 18 direction to out
```

```
$echo out > direction
```

```
$Turn LED on
```

```
$echo 1 > value
```

```
#Turn LED off
```

```
$echo 0 > value
```

In this example the LED is connected to GPIO pin 18. The LED can be connected to any other GPIO pin as well.

This figure shows how to manually control an LED connected to a Raspberry Pi using Linux terminal commands. The Raspberry Pi exposes its GPIO pins through special files in the `/sys/class/gpio/` directory. First, the user exports GPIO pin 18 so the operating system makes it available for use. Then, the pin direction is set to **out**, meaning the pin will send electrical signals instead of receiving them. To turn the LED ON, a value **1** is written to the pin, which sends a HIGH voltage to the LED. To switch it OFF, a value **0** is written, which sets the pin voltage LOW. These simple files write commands directly control physical hardware from the terminal, showing how easy it is to interface electronics with Raspberry Pi.

The below shows a Python program for blinking an LED connected to Raspberry Pi every second

Python program for blinking LED

```
import RPi.GPIO as GPIO
```

```
import time
```

```
GPIO.setmode(GPIO.BCM)
```

```
GPIO.setup(18, GPIO.OUT)
```

```
While True:
```

```
    GPIO.output(18, True)
```

```
    time.sleep(1)
```

```
    GPIO.output(18, False)
```

```
    time.sleep(1)
```

The program uses the RPi.GPIO module to control the GPIO on Raspberry Pi. In this program we set pin 18 directions to output and then write True/False alternatively after a delay of one second.

The figure shows a Python script that continuously blinks an LED using the Raspberry Pi GPIO library. The program imports the necessary modules, sets the GPIO numbering mode to BCM, and configures pin 18 as an output pin. Inside the infinite loop (while True:), the LED is turned ON by sending a HIGH signal, then the program waits for one second using time.sleep(1). After that, the LED is turned OFF by sending a LOW signal, followed again by a one-second delay. This process repeats continuously, making the LED blink. The example demonstrates how Python can be used to automate and control hardware in real time, making Raspberry Pi ideal for IoT and embedded applications.

Interfacing an LED and switch with Raspberry Pi

This example involves an LED and a switch that is used to control the LED. The schematic diagram of connecting an LED and switch to Raspberry Pi is as follows:

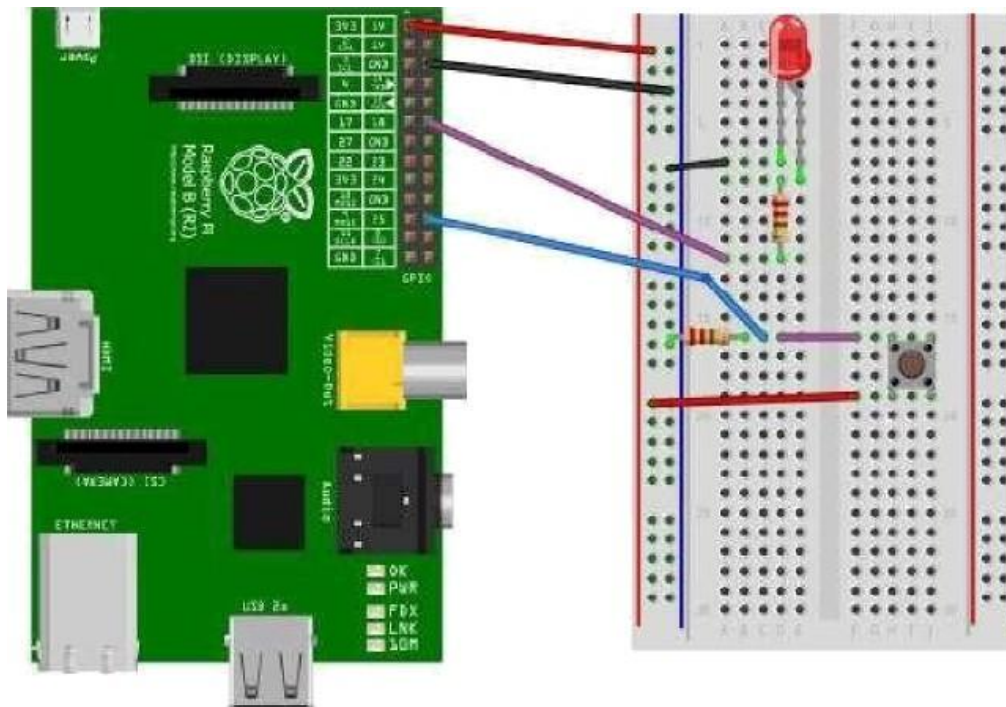


Figure 5 Interfacing an LED and Switch with Raspberry Pi

The Python program tier controlling an LED with a switch is as follows:

```
from time import sleep
import RPi.GPIO as GPIO

GPIO.setmode (GPIO.BCM)
#switch Pin
GPIO.setup(25, GPIO.IN)
#LED Pin
GPIO.setup(18, GPIO.OUT)

state = false

def toggleLED(pin):
    State = not state
    GPIO.output(pin, state)

While True:
    try:
        if (GPIO.input(25) == True):
            toggleLED (pin)
            sleep(.01)
    except KeyboardInterrupt :
        exit()
```

In this example the LED is connected to GPIO pin 18 and switch is connected to pin 25. In the infinite while loop the value of pin 25 is checked and the state of LED is toggled if the switch is pressed. it shows how to get input from GPIO pins and process the input and take some action. The action to this example is toggling the state of an LED.

Example: Another example, in which the action is an email alert. The Python program for sending an email on switch press is as follows:

Python program for sending an email on switch press

```

import smtplib
from time import sleep
import RPi.GPIO as GPIO
from sys import exit

from_email      = '<my-email>'
receipients_list = ['<receipient-email>']
cc_list         = []
subject         = 'Hello'
message         = 'Switch pressed on Raspberry PI'
username        = '<Gmail-Username>'
password        = '<password>'
server          = 'smtp.gmail.com:587'

GPIO.setmode(GPIO.BCM)
GPIO.setup(25, GPIO.IN)

def sendmail(from_addr, to_addr_list, cc_addr_list,
             subject, message, login, password, smtpserver):
    header = 'From: %s \n'%from_addr
    header += 'To: %s \n' %','.join(to_addr_list)
    header += 'Cc: %s \n' %','.join(cc_addr_list)
    header += 'Subject: %s \n \n' %subject
    message = header + message

    server = smtplib.SMTP(smtpserver)
    server.starttls()
    server.login(login,password)
    problems = server.sendmail(from_addr, to_addr_list, message)
    server.quit()

while True:
    try:
        if(GPIO.input(25) == True):
            sendmail(from_email, receipients_list,
                    cc_list, subject, message, username, password, server)
            sleep(.01)
    except KeyboardInterrupt:
        exit()

```

1. The program imports necessary libraries such as **smtplib** for email sending, **sleep** for timing, and **RPi.GPIO** for controlling Raspberry Pi GPIO pins.
2. Email configuration variables like sender address, recipient list, subject, message, Gmail username, password, and SMTP server are defined at the beginning.
3. The Raspberry Pi GPIO is set to **BCM mode**, and pin **25** is configured as an **input pin** to detect the switch press.
4. A function named **sendmail()** is created to generate and send email messages.
5. The function constructs email headers including “From”, “To”, “Cc”, and “Subject” fields, and combines them with the message body.
6. The script connects to Gmail’s SMTP server using port 587 and enables **TLS encryption** for secure communication.
7. It logs in to the Gmail account using the provided login credentials.
8. The email is sent to the specified recipient list using the **sendmail()** function of smtplib.
9. In the main loop, the program continuously checks the status of GPIO pin 25.
10. When the switch is pressed, the GPIO reads a HIGH signal and triggers the **sendmail()** function.
11. A short delay is added in the loop to prevent unnecessary CPU usage.
12. The program runs indefinitely until the user stops it manually, at which point it exits safely.
13. Overall, the script acts as a simple IoT alert mechanism that sends an email whenever the switch is pressed.

Interfacing a Light Sensor (LDR) with Raspberry Pi

In this we take up an example of interfacing a Light Dependent Resistor (LDR) with Raspberry Pi and turning an LED on/off based on the light-level sensed.

The schematic diagram of connecting an LDR to Raspberry Pi is presented as follows:

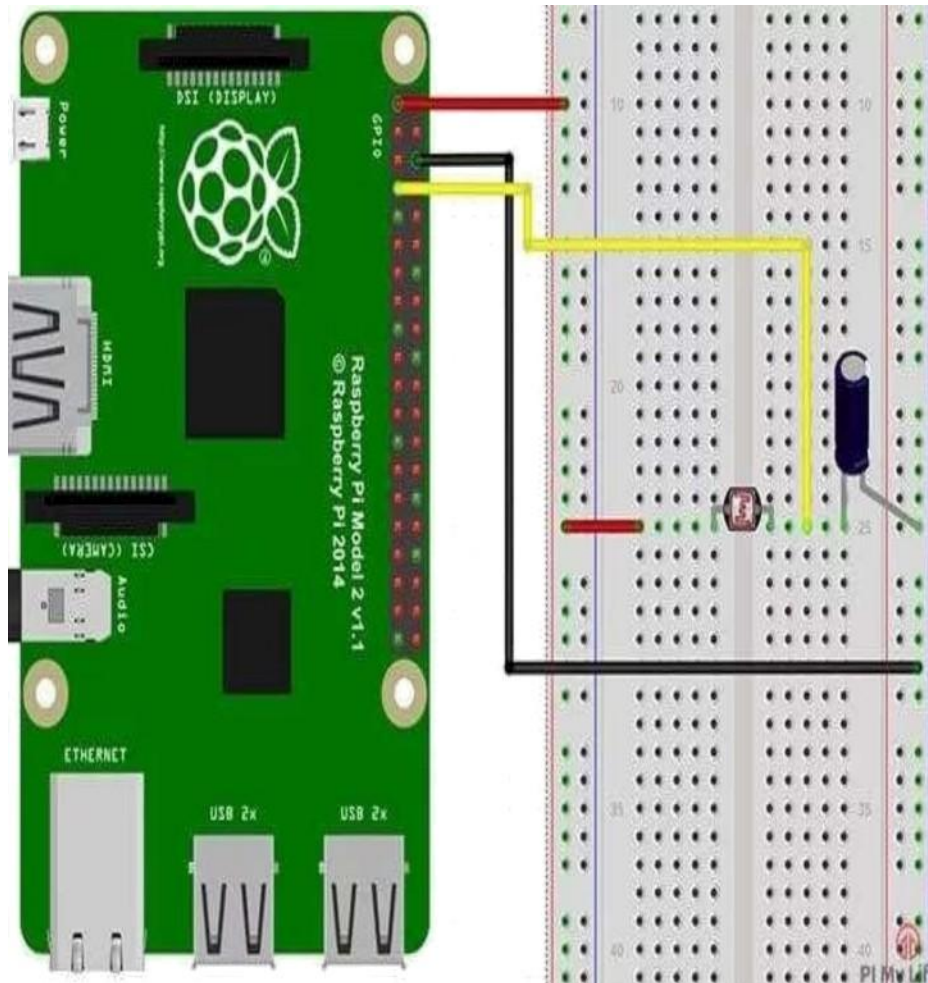


Fig: Interfacing a LDR with Raspberry PI

As shown in the figure the one side of LDR is connected to 3.3V and other side to a $1\mu\text{F}$ capacitor and also, to a GPIO pin (pin 18 in this example). An LED is connected to pin 18 which is controlled based on the light-level sensed.

Interfacing a Light Dependent Resistor (LDR) with a Raspberry Pi involves converting the analog behaviour of the sensor into a form that the digital GPIO pins can understand. An LDR's resistance decreases when exposed to light and increases in darkness. Since the Raspberry Pi does not have an in-built Analog-to-Digital Converter (ADC), the changing resistance of the LDR cannot be read directly by the GPIO pins.

To overcome this limitation, the LDR is usually connected as part of an external circuit that converts its resistance variation into a measurable electrical quantity such as voltage or time.

The most accurate and commonly used method is to connect the LDR in a voltage-divider circuit along with a fixed resistor and feed the output into an ADC like the MCP3008. In this setup, the LDR and resistor form a series connection between 3.3V and ground, and the voltage at their junction changes depending on the light intensity. This analog voltage is then read by the ADC, which converts it into a digital value that can be processed by the Raspberry Pi over the SPI communication interface. This method provides stable and precise readings suitable for applications where light levels need to be quantitatively monitored or logged.

A simpler alternative is the RC timing method, which does not require an ADC. Here, the LDR is paired with a capacitor, and the Raspberry Pi measures how long it takes for the capacitor to charge or discharge through the LDR. Because this time depends on the resistance of the LDR, it indirectly reflects the light intensity. Although this approach is inexpensive and easy to implement using just GPIO pins and a few passive components, the readings are relative and less accurate compared to an ADC-based system. It is mainly suitable for threshold-based applications such as turning on lights when the environment becomes dark.

Once the LDR readings are obtained, the Raspberry Pi can use them to make decisions in real time. For example, threshold values can be set to distinguish between bright, dim, and dark conditions. Applying smoothing or averaging techniques helps eliminate noise and ensures stable sensor behaviour. For more reliable control in automation scenarios, hysteresis can also be introduced—using different trigger points for turning a device ON and OFF—to prevent rapid switching in borderline lighting conditions.

Overall, interfacing an LDR with the Raspberry Pi allows the system to sense ambient light levels and act accordingly, enabling applications such as automatic lighting, display brightness control, and environmental monitoring. Whether using an ADC for precision or an RC circuit for simplicity, the LDR serves as a cost-effective sensor for a wide range of IoT and embedded system projects.

The python program for the LDR example is presented below. The readLDR() function returns a count which is proportional to the light level. In this function the LDR pin is set to output and low and then to input. At this point the capacitor starts charging through the resistor (and a counter is started) until the input pin reads high (this happens when capacitor voltage becomes greater than 1.4V). The counter is stopped when the input reads high. The final count is proportional to the light

level as greater the amount of light, smaller is the LDR resistance and greater is the time taken to

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
Ldr_threshold = 1000
LDR_PIN = 18
LIGHT_PIN = 25

def readLDR(PIN):
    reading = 0
    GPIO.setup(LIGHT_PIN, GPIO.OUT)
    GPIO.output(PIN, False)
    time.sleep(0.1)
    GPIO.setup(PIN, GPIO.IN)
    While (GPIO.input(PIN) == False):
        reading = reading+1
    return reading

def switchOnLight(PIN):
    GPIO.setup(PIN, GPIO.OUT)
    GPIO.output(PIN, True)

def switchOffLight(PIN):
    GPIO.setup(PIN, GPIO.OUT)
    GPIO.output(PIN, False)

While True:
    ldr_reading = readLDR(LDR_PIN)
    if ldr_reading < ldr_threshold:
        switchOnLight(LIGHT_PIN)
    else:
        switchOffLight(LIGHT_PIN)
    time.sleep(1)
```

charge the capacitor. **Python program for switching LED/Light based on reading LDR reading**

1. The program imports **RPi.GPIO** for pin control and **time** for delays.
2. GPIO mode is set to **BCM**, meaning pin numbers match the Broadcom chip numbering.
3. A threshold value for the LDR is defined to decide when the LED should turn ON or OFF.
4. Two GPIO pins are declared:
 - One for **LDR_PIN** (input)
 - One for **LIGHT_PIN** (output for LED)
5. The function **readLDR()** estimates the light intensity by timing how long it takes for the input pin to register a HIGH signal.
6. When the environment is **bright**, the reading value is **low**; when it is **dark**, the value is **high**.
7. The function **switchOnLight()** sets the LED pin as output and turns the LED **ON**.
8. The function **switchOffLight()** sets the same pin as output and turns the LED **OFF**.
9. Inside the infinite loop, the program continuously reads the LDR value.
10. If the LDR reading is **less than the threshold**, it indicates darkness, so the LED/light is switched **ON**.
11. If the reading is **greater than the threshold**, the surroundings are bright, so the LED is switched **OFF**.
12. A small delay is added for stable readings and to prevent rapid switching.
13. This program mimics an **automatic light control system**, commonly used in streetlights and smart home lighting.

Other IoT Devices

Some of the alternatives to Raspberry Pi are as follows:

1. **pcDuino:** It is an Arduino-pin compatible single board mini-computer that comes with a 1GHz ARM Cortex-A8 processor. It is a high performance and cost-effective device that runs PC like OS such as Ubuntu and Android ICS. Like, Raspberry Pi, it has an HDMI video/audio interface. It supports various programming languages including C, C++ (with GNC tool chain), Java (with standard Android SDK) and Python. The diagram is as follows:

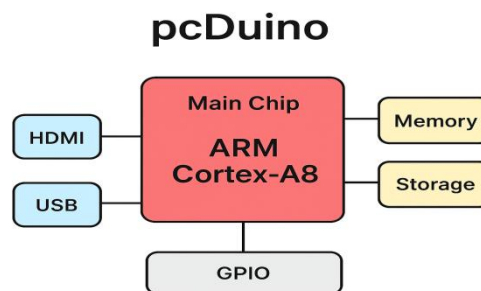


Fig: puDuino

1. ARM Cortex-A8 Processor (1GHz)

- The **main brain** of the pcDuino.
- Executes instructions, runs the operating system (Ubuntu/Android), and processes IoT applications.

2. Memory and Storage

- **DDR3 RAM** → Temporary working memory for fast execution.
- **Flash / SD Card Slot** → Permanent storage for operating system, programs, and IoT data.

3. Input/Output Interfaces

- **HDMI Port** → Connects to monitor/TV for video and audio output.
- **USB Ports (Type-A)** → For external devices (keyboard, mouse, USB storage, Wi-Fi dongle).
- **Micro USB Port** → Used for:
 - **Power supply** (if not using DC adapter).
 - **Uploading instructions/code** (when programmed like Arduino).
 - **Serial communication** with a PC.
- **Ethernet Port** → For wired internet access in IoT projects.

4. Arduino-Compatible GPIO Pins

- **GPIO (Digital I/O pins)** → Connects sensors, switches, actuators.
- **ADC (Analog-to-Digital Converter)** → Reads analog signals from sensors (temperature, light, etc.).
- **PWM (Pulse Width Modulation)** → Controls devices like servo motors, fan speed, or LED brightness.
- Arduino shield support makes it easy to extend hardware.

5. Power Management

- **DC Adapter / USB Power** → Provides 5V regulated supply.
- **PMU (Power Management Unit)** → Distributes stable power to all components.
- **Crystal Oscillator** → Maintains clock timing for the processor.

2. **BeagleBone Black:** It is similar to Raspberry Pi, but a more powerful device. It comes with a 1GHz ARM Cortex-A8 processor and supports both Linux and Android operating systems. Like Raspberry Pi, it has HDMI video/audio interface. USB and Ethernet ports. The diagram is as follows:

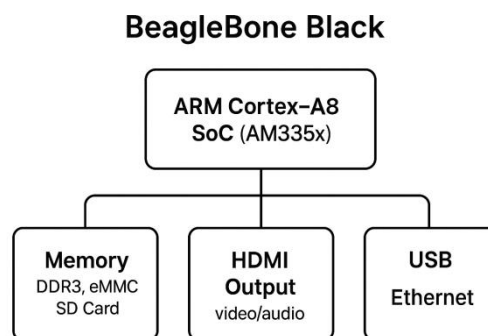


Fig: BeagleBone Black

1. ARM Cortex-A8 SoC (AM335x)

- This is the **main processing unit** of the BeagleBone Black.
- It handles all the computing tasks, runs Linux/Android operating systems, and manages input/output operations.
- It integrates CPU, GPU, timers, and other controllers into a single chip.

2. Memory (DDR3, eMMC, SD Card)

- **DDR3 RAM:** Provides working memory for fast operations.
- **eMMC Storage:** Built-in flash memory where OS and files can be stored.
- **SD Card Support:** Allows booting and additional storage expansion.
- Together, they store the system software, applications, and user data.

3. HDMI Output (Video/Audio)

- Provides interface to connect the BeagleBone Black with a monitor/TV.
- Supports both **video and audio output**, making it useful for multimedia IoT applications.

4. USB

- Used for connecting peripherals such as keyboard, mouse, Wi-Fi dongle, or USB storage.
- Also used for **power supply and programming** (uploading OS images or instructions).

5. Ethernet Port

- Provides wired network connectivity.
- Essential for IoT applications where stable, high-speed communication with servers, cloud, or other devices is required.

3. **Cubieboard:** It is powered by a dual core ARM Cortex A7 processor and has a range of input/output interfaces including USB, HDMI, IR, serial, Ethernet, SATA, and a 96-pin extended interface. It also provides SATA support. The board can run both Linux and Android operating systems. The diagram is as follows:

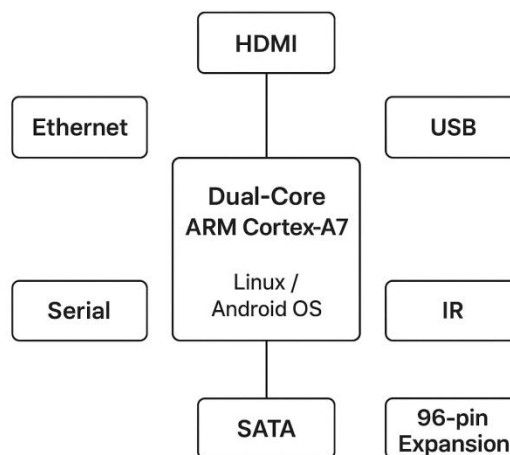


Fig: Cubieboard

1. Dual-Core ARM Cortex-A7 Processor (Center Block)

- The heart of the Cubieboard is the ARM Cortex-A7 dual-core processor.
- It is responsible for handling all computing tasks and coordinating input/output operations.
- Supports both Linux and Android OS, making it flexible for IoT and embedded applications.

2. HDMI Interface

- Provides video/audio output for connecting the Cubieboard to monitors, projectors, or TVs.
- Enables multimedia applications like video playback or GUI-based software.

3. USB Interface

- Allows connection to USB devices such as keyboards, mice, storage drives, and sensors.
- Acts as a major expansion option for IoT peripheral devices.

4. Ethernet Port

- Provides wired internet connectivity.
- Important for IoT applications requiring fast, stable network communication.

5. Serial Interface

- Useful for debugging and communication with other microcontrollers or hardware.
- Often used in IoT for device-to-device communication.

6. Infrared (IR) Interface

- Enables interaction with remote-control devices.
- Suitable for smart home and automation systems.

7. SATA Port

- Allows connection to external storage (like hard drives or SSDs).
- Useful for applications requiring large data storage (e.g., media servers, databases).

8. 96-Pin Extended Interface

- Provides additional GPIO pins and expansion capability.
- Enables interfacing with sensors, actuators, and custom IoT hardware modules.

The below table provides a comparison of some single-board mini-computers that can be used for IoT.

Name	Raspberry PI	Pc-Duino	Beagle Bone Black	Cubie Board
CPU	700 MHz ARM 11 Processor	1GHZ ARM Cortex A8	AM335x 1GHz ARM Cortex-A8	Dual Core 1GHz ARM Cortex-A7
GPU	Dual Core, Video Core IV Multimedia Co-Processor	Mali 400	Power VR SGX530	Dual Core ARM Mali 400 MP2
Memory	512MB	1GB	512MB	1GB
Storage	-	2GB Flash	2GB on-board	4GB NAND Flash
Networking	10/100M Ethernet	10/100M Ethernet	10/100M Ethernet	10/100M Ethernet
Input/ Output	2 USB, SD, MMC, SDIO card Slot	-	4+1 USB, Micro SDslot	2 USB, Micro SD, SATA, IR sensor
Interfaces	GPIO, SPI, I2C serial	Serial ADC, PWM, GPIO, I2C, SPI	69 pin GPIO, I2C, SPI, CAN, GPMC	96 extended pin interface
OS	Rasbian, Pidora, RISC OS, Arch, Linux	Ubuntu, Android	Ubuntu, Android, Linux, Angstrom	Android, official Linux Distribution
Video	HDMI, Composite RCA	HDMI	HDMI	HDMI
Audio	3.5 mm jack, HDMI	HDMI	HDMI	HDMI
Power	5VDC/700mA	5V/2A	5VDC/460mA	5VDC/2A

Note: Refer Lab Experiment Programs